Step 3 — A Retriever Agent That Judges Its Own Search Results — Hints

> Back to this step · Hint 1 · Hint 2 · Solution

Only 2 hints — work through them in order, and don't jump ahead until you've really tried. Each hint starts with the plain idea, then shows the shape of the real code.

- Hint 1 — Why "found something" isn't enough
- Hint 2 — The plan, and the final graph

> Back to this step · Hint 1 · Hint 2 · Solution

Hint 1 — Why "found something" isn't enough

The Retriever has three jobs:

1. **decide** whether to search at all (Doc10) — a keyword check: is the question about the documents' topics?
2. **search** — Doc08's `retrieve(collection, query, k)`, which returns dicts with `text`, `source_file`, `chunk_index` and `score`;
3. **judge** — a vector store *always* returns its k closest chunks, even for a question none of them answers. So "the list isn't empty" proves nothing.

The judgment is a number: `score` is a **distance** — lower means closer. Write the bar down *before* testing: "grounded if the best score is at most `MAX_DISTANCE`".

Bring Doc08's `chunking.py`, `ingest.py` and `retriever.py` into `rag/`. One change: create the `OpenAI()` client **inside** `build_vector_store()` and `retrieve()`, so the key is loaded first.

Build the store **once**: `rag/store.py` with a small holder class (like Doc07's `FlakyToolState`) and `get_collection()`, which builds on the first call and returns the same collection after that.

`agents/retriever_agent.py`:

- `route_search(state) -> "search" | "skip"`
- `is_relevant(sources) -> bool` — find the smallest score with a loop, compare to `MAX_DISTANCE`
- `retrieve_node(state)` → `{"sources", "grounded", "path"}`
- `has_grounding(state) -> "reason" | "no_grounding"`
- `no_grounding_node(state)` → an honest "couldn't find this" answer

> Back to this step · Hint 1 · Hint 2 · Solution

Hint 2 — The plan, and the final graph

```
START -> clean_task
clean_task  --route_search-->  "search": retrieve_node | "skip": answer_node
retrieve_node --has_grounding--> "reason": reason_node
                                 | "no_grounding": no_grounding_node
reason_node -> approval_node -> END
answer_node -> END          (general answer, labelled)
no_grounding_node -> END
```

This is the graph's final shape — Steps 4 and 5 won't touch `graph.py`.

`reason_node` lives in its own file, `agents/reasoner_agent.py`, from the start. For now it needs **no model**: `answer = best["text"] + " [" + best["source_file"] + "]"`. It proves search → answer → approval works before Step 4 upgrades the inside of this same node.

Documents: 5 short `.md` files in `data/docs/`, plain paragraphs, **no heading lines** — a lone heading becomes its own chunk and matches random questions.

Tests, one function per rule, each printing `OK` or `WRONG`: `test_routing.py` with hand-made states and score lists (`is_relevant([{"score": 0.4}])` → `True`); `test_chunking.py` ; and integration cases 5 (a document question is grounded) and 6 (an unknown topic ends at `no_grounding_node`). To tune `MAX_DISTANCE` , read the `found ... (score ...)` log lines for good and bad questions.

Full solution: Show me the solution